

Jour 5 : TCP Handshake & TCB

Le Transport Control Protocol (TCP) est beaucoup plus complexe que l'UDP car il est **orienté connexion** et garantit la livraison des données.

1. Le TCP Control Block (TCB)

Contrairement à l'IP ou l'ICMP qui sont "sans état" (stateless), TCP doit se souvenir de chaque connexion. Pour cela, on utilise une structure appelée **TCB**. Elle stocke :

- Les ports et IPs de la connexion.
- Les numéros de séquence (SEQ) et d'acquittement (ACK).
- L'état actuel (LF : LISTEN, SYN_SENT, ESTABLISHED, etc.).

2. Le 3-Way Handshake (La poignée de main)

Pour établir une connexion, TCP utilise trois étapes :

1. **SYN** : Le client choisit un numéro de séquence aléatoire ISN_c et l'envoie.
2. **SYN-ACK** : Le serveur accepte, choisit son propre ISN_s et répond en acquittant le client : $ACK = ISN_c + 1$.
3. **ACK** : Le client termine en acquittant le serveur : $ACK = ISN_s + 1$.



```
root@kali: /home/kali/5_jour
Session Actions Edit View Help
root@kali: /home/kali/5_jour  kali@kali: ~  kali@kali: ~
(kali@kali)~[~]
$ sudo su
[sudo] password for kali:
(root@kali)~/home/kali
$ cd 5_jour
(root@kali)~/home/kali/5_jour
$ chmod 777 *
(root@kali)~/home/kali/5_jour
$ gcc *.c -o main
(root@kali)~/home/kali/5_jour
$ ./setup_net.sh
net.ipv6.conf.tap0.disable_ipv6 = 1
Interface tap0 configurée sur 10.0.0.1. Votre programme doit utiliser 10.0.0.2
(root@kali)~/home/kali/5_jour
$ ./main
Interface tap0 ouverte (fd = 3)
5: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether fe:fb:f0:57:0d:7f brd ff:ff:ff:ff:ff:ff
```

ensuite on lance une connexion nc sur l'adresse du programme c'est à dire 10.0.0.2 dans notre cas sur le port 80 par exmple.

```
root@kali: /home/kali/5_jour
Session Actions Edit View Help
root@kali: /home/kali/5_jour kali@kali: ~ kali@kali: ~
Interface tap0 ouverte (fd = 3)
5: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether fe:fb:f0:57:0d:7f brd ff:ff:ff:ff:ff:ff

== Trame reçue (42 octets) ==
Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : fe:fb:f0:57:0d:7f
EtherType : 0x0806 → ARP

--- Cache ARP Interne ---
[0] 10.0.0.1 → fe:fb:f0:57:0d:7f

ARP Reply envoyé à 10.0.0.2

== Trame reçue (74 octets) ==
Dst MAC : 00:11:22:33:44:55
Src MAC : fe:fb:f0:57:0d:7f
EtherType : 0x0800 → IPv4

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 60
TCP: Paquet reçu sur port 80 (Source: 58470)
TCP: Requête SYN reçue - Création d'une connexion (Handshake)
TCP: SYN-ACK envoyé à 10.0.0.1:58470

== Trame reçue (54 octets) ==
Dst MAC : 00:11:22:33:44:55
Src MAC : fe:fb:f0:57:0d:7f
EtherType : 0x0800 → IPv4

--- Paquet IP reçu ---
Source: 10.0.0.1
Destination: 10.0.0.2
Protocol: 6
Total Length: 40
TCP: Paquet reçu sur port 80 (Source: 58470)
TCP: ACK reçu - Connexion ESTABLISHED !
```

comme on voit , la connexion est établie donc essayer d'envoyé des messages vous verrez le transfert des paquets

3. Focus Sécurité : SYN Flood Attack

C'est une attaque de Déni de Service (DoS) qui exploite la file d'attente des connexions "semi-ouvertes".

- **L'attaque** : L'attaquant envoie des milliers de paquets **SYN** avec des IPs sources falsifiées.
- **Le problème** : Pour chaque SYN, le serveur alloue un **TCB** en mémoire et attend un ACK qui ne viendra jamais.
- **Résultat** : La table des TCB sature. Les utilisateurs légitimes ne peuvent plus se connecter car le serveur "consomme" ses ressources pour des connexions fantômes.